

Daniel Alejandro Barrientos

Tegucigalpa, Honduras

danielbanariba@protonmail.com • (+504) 3324-5827

danielbanariba.dev • github.com/danielbanariba • linkedin.com/in/danielbanariba

Ingeniero en Sistemas y Data Engineer. Construyo herramientas para que la IA opere infraestructura de producción: un fleet propio de servidores MCP operado desde Claude Code, y pipelines ETL con Dagster/Python sobre un DWH en SQL Server con monitoreo autónomo y alertas multicanal. Optimicé un forecast mensual de ~18h a ~61 min paralelizando con multiprocessing.

EXPERIENCIA LABORAL

Grupo Farinter

Desarrollador RPA & Data Engineer | Noviembre 2025 - Actualidad

- Optimicé un forecast mensual de demanda (~23,000 series) de ~18 horas a ~61 minutos en producción, paralelizando con ProcessPoolExecutor (poda de series muertas, output byte-idéntico y determinista) con TDD estricto y sin dependencias nuevas.
- Construí y opero un fleet propio de servidores MCP (Model Context Protocol) —Dagster, SQL Server, SMB, Dokploy, Microsoft Graph/Outlook— con validador read-only, optimización de tokens (~10-20x) y auditoría adversarial multiagente, operado desde Claude Code.
- Desarrollé pipelines de conciliación bancaria (Credomatic/BAC, Ficohsa) y transaccional (Netflix, Tengo, Rapibac) vía SMB y SQL Server, con detección de archivos por file sensors y carga incremental.
- Construí un dashboard full-stack en Python (Reflex) que compara los márgenes de Kielsa contra ~7,700 SKUs de competencia en 8 vistas analíticas, con detección de erosión de margen (SCD2) y simulación de precio sugerido; 801 tests y CI/CD a Dokploy.
- Construí un sistema autónomo (watchdog) que detecta pipelines pegados, los re-lanza automáticamente y notifica vía Telegram, complementado con timeouts nativos (dagster/max_runtime) en 27+ jobs.
- Cerré un hueco de calidad de datos con un sensor genérico de AssetCheckEvaluation que notifica solo errores a Discord/Telegram/email (dedup por cursor); destapó un duplicado silencioso de 182,928 filas que pasaba con los runs en verde.
- Construí scrapers de precios de competencia con extracción diaria en Dagster, descriptación de APIs y rotación de proxies/TOR.
- Apliqué Agentic AI con orquestación de sub-agents, Anthropic Agent Skills (SKILL.md), slash commands y hooks sobre Claude Code, Codex CLI y OpenCode, bajo metodología SDD y TDD estricto.
- Construí hvault, servicio en Rust puro (CLI + daemon) que sincroniza certificados TLS de Traefik entre nodos vía HashiCorp Vault (KV-v2), con escritura atómica para hot-reload sin downtime, validación x509 y UI de administración en Leptos/WASM con OIDC.
- Automaticé cierre de cajas, migré procesos legacy de Windows a Dagster y estandaricé tablas DWH con metadatos (país, origen, versiones) para consumo en Power BI.

Tecnologías: Python, Dagster, SQL Server, DWH, Docker, SMB, Discord Webhooks, dbt, MCP, Model Context Protocol, Claude Code, Codex CLI, OpenCode, Cursor, Aider, Cline, Continue.dev, Windsurf, Gemini CLI, GitHub Copilot, Agent Skills, SKILL.md, Slash Commands, Hooks, AI Agents, Agentic AI, Sub-agents, Agent Orchestration, Tool Use, Function Calling, AI Integration, LLMs, GenAI, Anthropic API, OpenAI API, SDD, Spec-Driven Development, TDD, Prompt Engineering, Git

Analiza Laboratorios Clínicos

Desarrollador de Software | Marzo 2025 - Octubre 2025

- Automaticé integración VITEK-CRM con FastAPI y protocolos ASTM/HL7 en producción para procesamiento bidireccional de resultados de cultivos.
- Modelé catálogos con SQLAlchemy ORM (7,429 microorganismos, 278 antibióticos) con relaciones complejas entre encabezados y detalles de antibiogramas.
- Creé proxy corporativo con Docker, Squid y OpenVPN reduciendo gestión de 30+ equipos a 1.

Tecnologías: Python, FastAPI, TypeScript, NestJS, PostgreSQL, SQLAlchemy, TypeORM, Docker, WebSocket, PowerShell, ASTM, HL7, VITEK

GuabaBIT

QA & Backend Developer | Octubre 2024 - Marzo 2025

- Diseñé documentación técnica de arquitectura completa para sistema de pagos comunitarios con servicios RESTful, NestJS, DynamoDB y servicios AWS (S3, SNS).

- Implementé pruebas E2E con Playwright para flujos críticos (login, formularios, cotizaciones, handshake) y automaticé testing de API con Postman/Apidog.

Tecnologías: TypeScript, NestJS, DynamoDB, AWS (S3, SNS), Playwright, Jest, Postman, Apidog, Jira, Swagger, Microservicios, Serverless

HABILIDADES

- Lenguajes: Python, TypeScript, JavaScript, C++, Rust, SQL, T-SQL
- IA & Desarrollo Asistido por IA: Claude Code, Cursor, GitHub Copilot, MCP (Model Context Protocol), Anthropic Agent Skills, AI Agents, Sub-agents, Tool Use / Function Calling, Prompt Engineering, SDD (Spec-Driven Development), Anthropic API, LLMs / GenAI
- Frameworks & Orquestación: Dagster, FastAPI, NestJS, Reflex, Astro
- Bases de Datos: SQL Server, PostgreSQL, MongoDB, DynamoDB, SQLite
- Cloud & Infraestructura: AWS (S3, SNS, DynamoDB), Azure, Docker, Terraform, Dokploy, HashiCorp Vault
- Testing & QA: Playwright, Jest, Postman, Apidog, Jira
- Data Engineering: dbt, Power BI, ETL, Web Scraping, DWH, Polars, statsforecast, Data Quality

PROYECTOS

Navidrome — Contribuidor Open Source

Contribuidor open-source con contribuciones mergeadas al servidor de streaming musical Navidrome (20.7K ■ en GitHub), en frontend (react-admin) y accesibilidad web.

Tecnologías: React, react-admin, JavaScript, Open Source

MCP Servers Suite — Fleet propio en producción

Fleet de servidores MCP (Model Context Protocol) en producción operados desde Claude Code: mcp-dagster (assets/runs vía GraphQL), mcp-db (queries con Jinja + validador read-only sobre SQL Server/PostgreSQL), mcp-api-check (health checks), smb-mcp (archivos sobre SMB), mcp-dokploy (despliegues) y outlook-todoist-mcp (Microsoft Graph). Modernizado con optimización de tokens (~10-20x), auditoría adversarial multiagente y split de monolitos.

Tecnologías: Python, MCP, Dagster GraphQL, SQL Server, PostgreSQL, Microsoft Graph, smbprotocol, Dokploy, Claude Code

Paralelización de Forecast Dagster — 18h → 61 min

Optimización de un forecast mensual de demanda (~23,000 series con statsforecast) que corría ~18h en 1 core y se mataba contra el cap de runtime. Reimplementado con ProcessPoolExecutor, pin de threads BLAS, detección de CPUs cgroup-aware y poda de series muertas (~31.6%); resultado ~61 min con output byte-idéntico y determinista. SDD + TDD estricto (30 tests), solo stdlib.

Tecnologías: Python, statsforecast, concurrent.futures, ProcessPoolExecutor, Dagster, SQL Server, SDD, TDD, pytest

EDUCACIÓN

Universidad Nacional Autónoma de Honduras

Licenciatura en Matemáticas (en curso) | Tegucigalpa, Honduras | 2024 - Actualidad

Universidad Nacional Autónoma de Honduras

Ingeniería en Sistemas Computacionales | Tegucigalpa, Honduras | 2019 - 2025

CERTIFICACIONES

- Oracle Next Education F2 T5 Back-end - Alura Latam, 2023
- Database Foundations - Oracle Academy, 2023

IDIOMAS

- Español: Nativo
- Inglés: Nivel B2